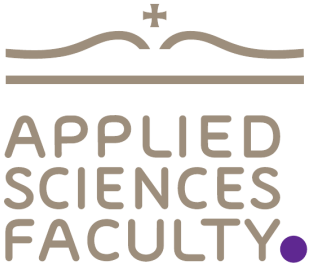
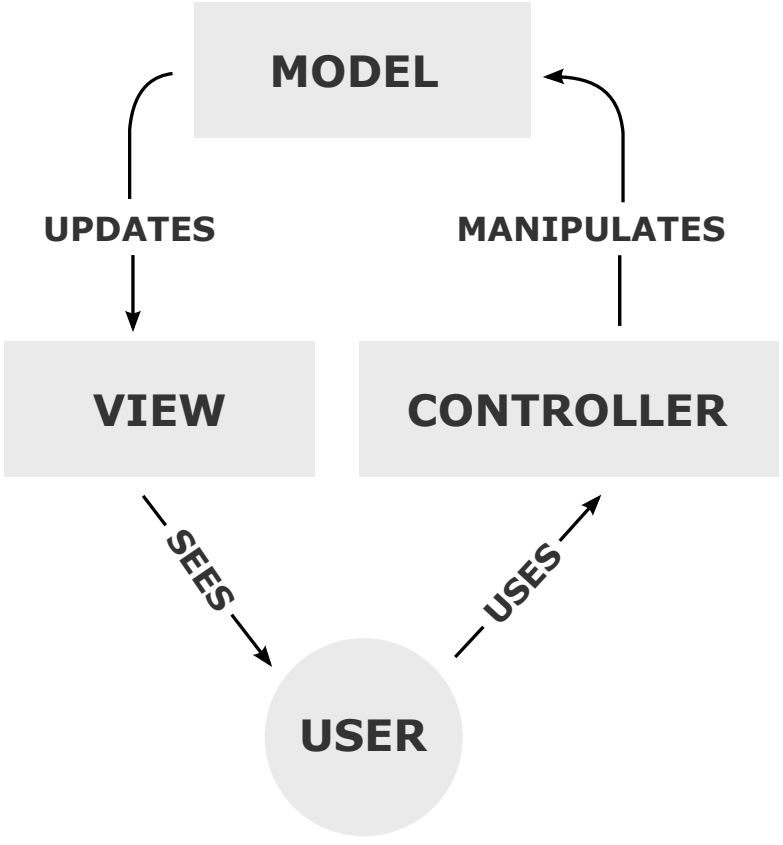
C++ — додаток. Qt. Lecture XX — MVC





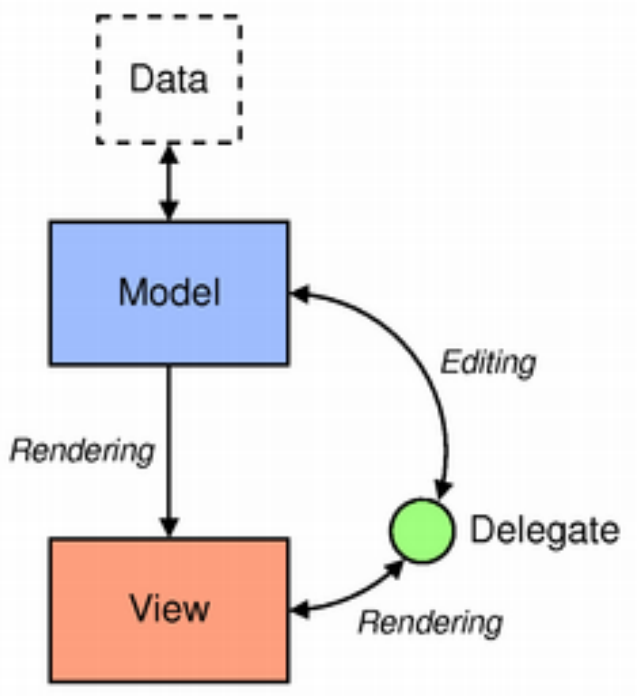
©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

MVC — Model-View-Controller



- Модель — те, що реалізовує поведінку програми, безвідносно до її інтерфейсу.
- Представлення, view — запитує інформацію в моделі, для відображення її користувачеві.
- Контролер — взаємодіє із користувачем, як змінюючи модель, (редагування даних) так і її вигляд, представлення (прокрутка).

Qt Model-View



- Qt об'єднує контролер та представлення, тому можна говорити про **Model-View** архітектуру.
- Рудиментом Controller можна вважати **делегати**.
- Всі ці три сутності представлені абстрактними класами, які задають інтерфейси. Для взаємодії використовуються сигнали та слоти.
- Модель Qt – оболонка навколо вихідних даних. Дані можуть як зберігатися якимось чином, так і обчислюватися на льоту.

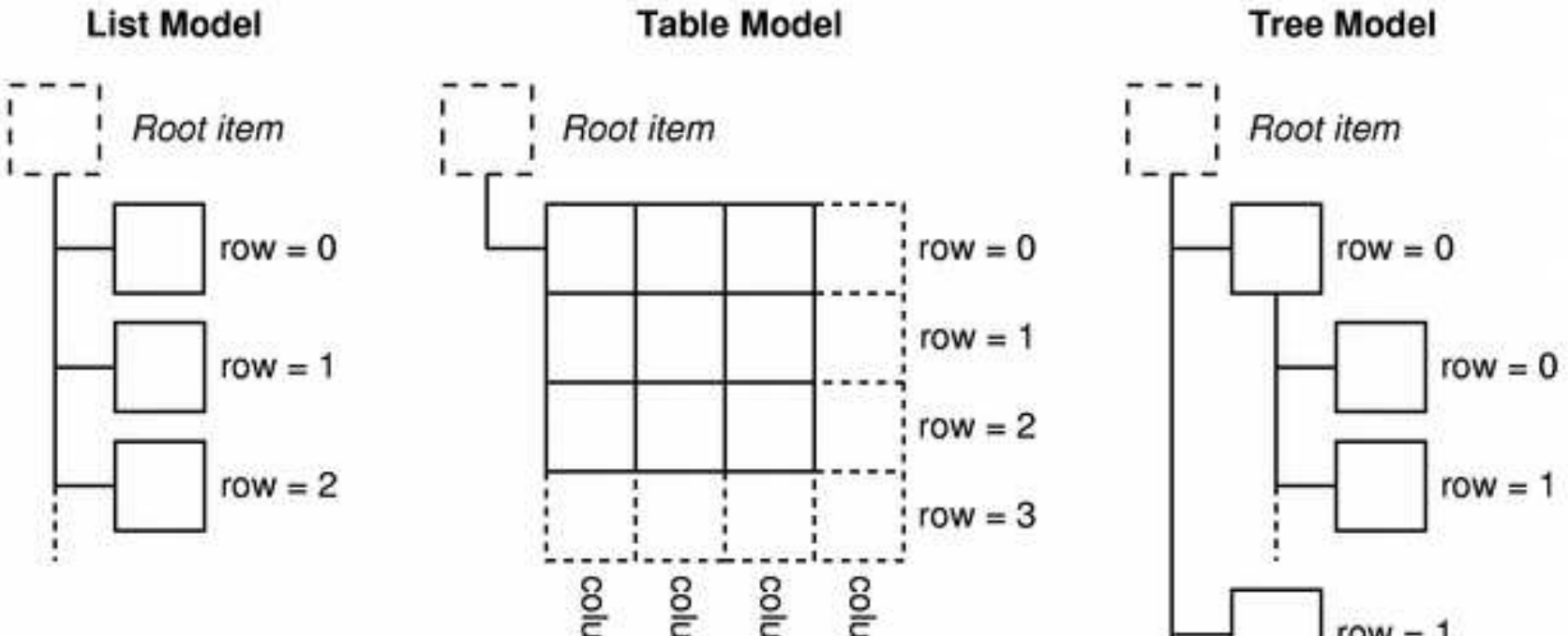
Модель з точки зору Qt

Базовий клас:

- QAbstractItemModel — узагальнена абстракція таблиці, кожним елементом якої може бути інша таблиця. Її більш спеціалізовані нащадки реалізовують частину чи всі чисто віртуальні функції.
- Сигнали моделі повідомляють представлення (view) про зміни в даних.

Абстрактні нащадки:

- QAbstractTableModel — модель двовимірної таблиці.
- QAbstractListModel — модель для роботи із абстрактним списком.



Конкретні загальні моделі:

- QStringListModel — нащадок QAbstractListModel, модель для роботи із списками стрічок, джерелом даних служить QStringList, (зміни моделі можуть змінювати вихідний список).
- QStandardItemModel — модель, яка містить дані безпосередньо в собі.

Проксі-моделі:

- QAbstractProxyModel використовується для того, щоб витягнути певні дані з моделі, опрацювати їх (наприклад, відфільтрувати чи посортувати) і представити як нову модель.
- Безпосередньо для сортування та фільтрування використовується її нащадок, QSortFilterProxyModel.

Спеціалізовані моделі:

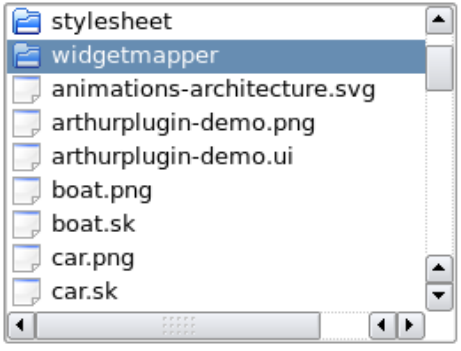
- QFileSystemModel[1] — модель, що представляє файлову систему.
- QSqlQueryModel, QSqlTableModel, QSqlRelationalTableModel – моделі для доступу до баз даних.

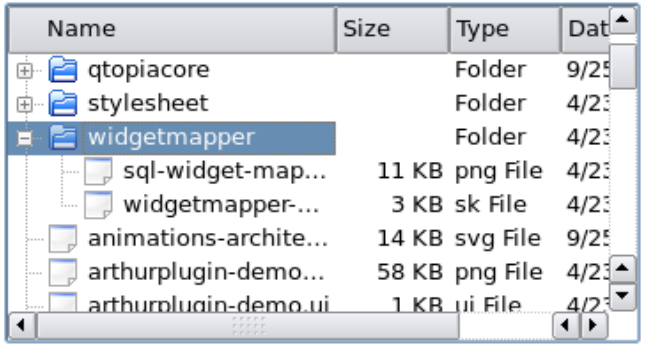
[1] QDirModel – застарілий варіант.

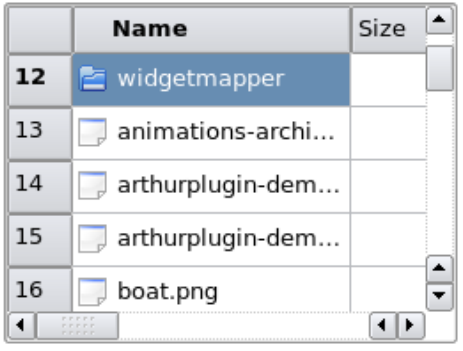
Представлення в Qt

- Для представлень Qt надає дві ієрархії засобів. Головна – це нащадки QAbstractItemView. Крім того, QComboBox теж може служити для відображення моделі.
- QAbstractItemView є нащадком QAbstractScrollArea, тому підтримує можливість скроллінгу (прокрутки).
- Нащадками абстрактного представлення є QListView, QTableView, QTreeView, QColumnView, QHeaderView.

- QListView, QTableView, QTreeView реалізовують три різних концепції відображення:







- QHeaderView служить для представлення заголовків, як горизонтальних, так і вертикальних, в QTableView і QTreeView.
- Керувати ними можна за допомогою методу QAbstractItemModel::setHeaderData().
- Кожен заголовок має орієнтацію: Qt::Horizontal або Qt::Vertical, **роль** – див. далі, та, власне, вміст, що відображатиметься.
- Увага!, така проста модель, як QStringListModel, не передбачає засобів для збереження заголовків!
- Приклад:

  ```
  model->setHeaderData(0, Qt::Horizontal, "Column 1", Qt::DisplayRole);
  ```

- QColumnView служить для представлення одного стовпця моделі.

Виділення елеметнів

- Виділенням керує клас QItemSelectionModel.
- Абстрактна модель QAbstractModelView дозволяє встановити модель виділення за допомогою методу setSelectioModel().
- Виділенням можна керувати програмно, за допомогою select(), а отримати вибрані елементи за допомогою selectedIndexes().
- Про зміни виділення та поточного елемента, повідомляють відповідні сигнали: currentChanged(), currentColumnChanged(), currentRowChanged(), selectionChanged().

Делегати

- Відображенням та редагуванням елементів представлення займаються делегати.
- Предковим класом для делегатів служить QAbstractItemDelegate.
- По замовчуванню використовується QItemDelegate, який надає для редагування просте текстове поле. За потреби у якихось складніших засобах редагування, можна створювати свої делегати.

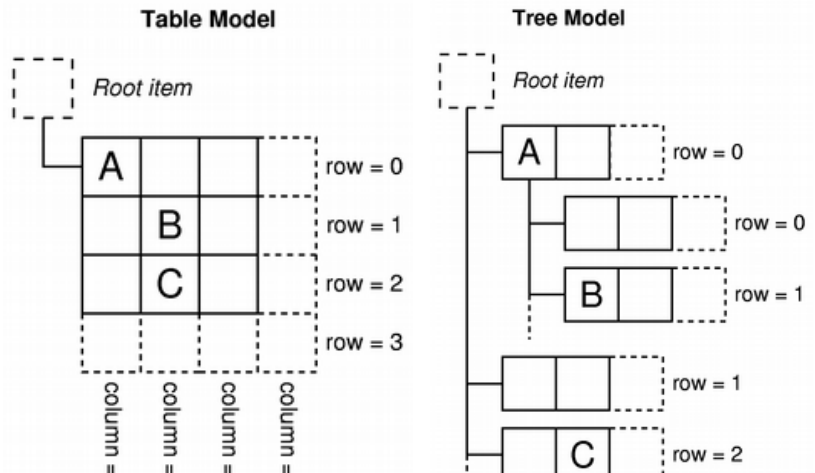
Індекс моделі

- Індекс моделі — унікальний номер комірки.
- Реалізовується класом QModelIndex, який зберігає рядок, стовпець та її предка і вказівник на "свою" модель.
- Для таблиці (чи списку) все просто:

```
QModelIndex indexA = model->index(0, 0, QModelIndex());
QModelIndex indexB = model->index(1, 1, QModelIndex());
QModelIndex indexC = model->index(2, 1, QModelIndex());
```

- Але для дерева потрібно також вказувати вузол, відносно якого задано рядок та стовпець:

```
QModelIndex indexA = model->index(0, 0, QModelIndex());
QModelIndex indexC = model->index(2, 1, QModelIndex());
QModelIndex indexB = model->index(1, 0, indexA);
```



Індекс моделі

- Дані, що відповідають комірці, можна отримати, за допомогою QAbstractItemModel::data():

  QVariant QAbstractItemModel::data(**const** QModelIndex &index,
  int role = Qt::DisplayRole) **const**

- Вказівник на модель, на яку вказує індекс – методом QModelIndex::model();
- Отримати індекс для будь-якої комірки можна за допомогою QAbstractItemModel::index(). Якщо такого елемента немає, повертається порожній індекс. Перевірити, чи індекс вказує на елемент, можна за допомогою його методу isValid().
- QModelIndex може інвалідуватися при перебудові моделі. **QPersistentModelIndex** — індекс, який можна зберігати в програмі (замість отримувати кожен раз наново). QPersistentModelIndex конструюється із QModelIndex, однак, перед використанням варто завжди перевіряти його валідність.

Ролі

- DisplayRole — для відображення в формі тексту
- DecorationRole — в формі зображення
- EditRole — форма, зручна для редагування в текстовому редакторі
- ToolTipRole — підказка
- StatusTipRole — підказка в стрічці статусу
- WhatsThisRole — для виведення в "What's This?"
- SizeHintRole — підказка щодо розміру елементу, яка буде передаватися представленню.
- Ролі, які стосуються вигляду, для делегатів: FontRole, TextAlignmentRole, BackgroundRole, BackgroundColorRole, ForegroundRole, (deprecated TextColorRole), CheckStateRole, InitialSortOrderRole.
- Додатково, для підтримки виводу для людей із особливими потребами: AccessibleTextRole і AccessibleDescriptionRole.
- UserRole – роль, від якої відраховуються користувацькі ролі.

QVariant — об'єднання, що може містити більшість типів Qt

Створення власних моделей — 1

- Для того, щоб створити модель, слід реалізувати цілий набір чисто віртуальних функцій з QAbstractItemModel.
- Для спрощення реалізації типового списку і таблиці, можна скористатися QAbstractListModel чи QAbstractTableModel, які для типових функцій мають адекватні реалізації по замовчуванню.
- Якщо модель надає дані тільки для читання, слід перевизначити: flags(), data(), headerData(), rowCount(). columnCount() слід також реалізувати для всіх, крім QAbstractListModel.
- Для можливості редагування, модель повинна надавати: flags(), яка повертає Qt::ItemIsEditable для елементів, які можна редагувати, setData(), setHeaderData().
- Для моделей, розмір яких можна змінювати, слід реалізувати: insertRows(), insertRows(), insertColumns(), removeColumns().

Створення власних моделей — 2

- Модель повинна інформувати про зміни у ній, щоб можна було оновити пов'язані із нею представлення. Зокрема, data() повинен генерувати сигнал dataChanged(), setHeaderData() — сигнал headerDataChanged(). За радикальної зміни моделі, слід перед початком викликати beginResetModel(), завершивши – endResetModel(). insertRows() повинна викликати beginInsertRows() та endInsertRows(), це ж стосується і решти: insertRows(), insertColumns(), removeColumns(). За зміни структури моделі (реорганізація, сортування, тощо), слід генерувати сигнал layoutChanged(), перед початком змін слід згенерувати layoutAboutToBeChanged().
- Це ж стосується й інших методів моделі, які її змінюють.

QDataWidgetMapper

- Клас, що дозволяє відображати вміст моделі на віджети. Приклад використання — одна частина вікна це таблиця, а друга – детальна інформація щодо вибраного рядка.
- Відображаються так звані секції моделі, скажімо, різні віджети можуть відповідати різним стовпцям одного рядка (або навпаки).
- Приклад із офіційної документації:

```
QDataWidgetMapper *mapper = new QDataWidgetMapper;
mapper->setModel(model);
mapper->addMapping(mySpinBox, 0);
mapper->addMapping(myLineEdit, 1);
mapper->addMapping(myCountryChooser, 2);
mapper->toFirst();
```

  Стовпчик 0 моделі містить число, 1 – текст, 2 – країну із списку. Вони відображаються на відповідні віджети (mySpinBox, myLineEdit, myCountryChooser). toFirst() переходить до першого рядка моделі, вибираючи дані саме з нього.
- Є також toFirst(), toNext(), toPrevious(), toLast(), setCurrentIndex().
- Є два режими повернення даних назад в модель: AutoSubmit та ManualSubmit. Перший поміщає дані зразу після того, як віджет втратив фокус, другий - по виклику submit().

Додаткові можливості

- Drag-and-Drop
- ”Ліниве” підвантаження даних.

Посилання

- `http://doc.qt.io/qt-5/model-view-programming.html` — офіційний підручник по MVC в Qt. Детальний та доступний, дуже рекомендую! Див. також посилання там, зокрема — на конкретні класи та приклади (в кінці).
- Приклад, який напряму важко знати: `http://qt-project.org/doc/qt-4.8/itemviews-addressbook.html` — Address Book Example. Список інших аналогічних прикладів: `http://qt-project.org/doc/qt-4.8/examples-itemviews.html` — "Item Views Examples".
- `http://www.bogotobogo.com/Qt/Qt5_QListView_QStringListModel_ModelView_MVC.php` та інші приклади до MVC там же.
- `https://uk.wikipedia.org/wiki/Model-View-Presenter` — варіант MVC.
- `http://doc.qt.io/qt-5/qdatawidgetmapper.html` — відображення моделей на віджети.